

SMART AD OPTIMIZATION SYSTEM

Project Submitted to the
SRM University AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of
Bachelor of Technology
in
Computer Science & Engineering
School of Engineering & Sciences

submitted by
Snehitha Gandu (AP24110011008)
Shahistha Anjum (AP24110010993)



Department of Computer Science & Engineering

SRM University-AP

Neerukonda, Mangalgiri, Guntur

Andhra Pradesh- 522 240

April 2025

ABSTRACT :

The Smart Ad Optimization System is a computational model developed to address budget allocation challenges in digital advertising. In modern marketing environments, businesses must choose from multiple advertising opportunities while operating under strict financial constraints. Selecting the most profitable combination manually is inefficient and often leads to suboptimal decisions.

This project introduces an algorithm-driven approach to solve this problem using two widely studied techniques: the Greedy Algorithm and Dynamic Programming (0/1 Knapsack Problem). The system evaluates advertisements based on their cost and expected profit, and determines the best subset that maximizes total profit without exceeding the available budget.

By implementing both approaches, the system not only provides optimized solutions but also enables a comparative analysis between a fast heuristic method (Greedy) and a guaranteed optimal method (Dynamic Programming). This highlights the trade-offs between computational efficiency and solution accuracy, making the project both practical and educational.

INTRODUCTION :

In today's highly competitive digital landscape, advertising plays a critical role in business growth and customer acquisition. Companies allocate significant budgets to various advertising platforms such as social media, search engines, and display networks. However, the challenge lies in optimally distributing this limited budget across multiple advertising options.

Each advertisement varies in:

- Cost (investment required)
- Expected return (profit or engagement)

Choosing the right combination of advertisements becomes a complex decision-making problem, especially when the number of options increases. Traditional decision-making methods are often based on intuition or incomplete analysis, which may lead to inefficient budget utilization.

This project addresses this issue by modeling the problem as an optimization problem and solving it using algorithmic techniques from the field of Design and Analysis of Algorithms (DAA).

The system is designed to:

- Accept user-defined advertisement data
- Process inputs using algorithmic strategies
- Provide optimized outputs based on constraints
- Compare different algorithmic approaches

By doing so, the project bridges the gap between theoretical algorithm concepts and real-world applications in digital marketing.

OBJECTIVES :

The primary objective of the Smart Ad Optimization System is to develop a practical application that demonstrates how algorithmic techniques can be used to solve real-world optimization problems. The project aims to implement the Greedy Algorithm to provide a fast and efficient

method for selecting advertisements based on their profit-to-cost ratio. In addition, it focuses on demonstrating the use of Dynamic Programming, specifically the 0/1 Knapsack approach, to ensure an optimal selection of advertisements under given constraints.

Another important objective is to compare both approaches in terms of efficiency and optimality. While the Greedy method is computationally faster, it may not always produce the best solution, whereas Dynamic Programming guarantees optimal results at the cost of higher computation time. Through this comparison, the project highlights the importance of choosing the right algorithm for different scenarios. Finally, the project aims to build a fully functional optimization system using the C programming language, incorporating core concepts such as structures, arrays, sorting, and memory management.

PROBLEM DEFINITION :

The problem addressed in this project is a classic optimization problem commonly encountered in digital marketing and resource allocation. Given a set of advertisements, each advertisement is associated with two key parameters: cost, which represents the amount of budget required to run the ad, and profit, which represents the expected return or benefit gained from that advertisement.

The system operates under a strict constraint: the total cost of selected advertisements must not exceed the available budget. Within this limitation, the goal is to determine the best possible combination of advertisements that maximizes the total profit. Since multiple combinations of ads are possible, selecting the optimal subset becomes computationally challenging, especially as the number of advertisements increases. This problem is effectively modeled and solved using algorithmic approaches such as Greedy and Dynamic Programming.

SYSTEM DESIGN :

The Smart Ad Optimization System is designed to follow a structured workflow consisting of three main stages: input, processing, and output.

In the input stage, the system collects all necessary data from the user. This includes the number of advertisements, along with the details of each advertisement such as its name, cost, and expected profit. Additionally, the user provides the total available budget, which acts as the primary constraint for the optimization process.

In the processing stage, the system performs calculations and applies algorithmic logic to determine the best selection of advertisements. Initially, it computes the profit-to-cost ratio for each advertisement, which is used in the Greedy approach. The system then applies two different algorithms: the Greedy method, which selects ads based on the highest ratio, and the Dynamic Programming method, which evaluates all possible combinations to find the optimal solution.

In the output stage, the system displays the results obtained from both approaches. This includes the list of selected advertisements and the total profit generated for each method. By presenting both results, the system allows users to clearly observe the difference between a fast heuristic solution and an optimal solution.

ALGORITHM DESIGN :

The Greedy Algorithm is designed to make the best possible choice at each step with the aim of achieving a globally optimal solution. In this project, the algorithm begins by calculating the profit-to-cost ratio for each advertisement. This ratio represents the efficiency of each ad in terms of profit generated per unit cost.

Once the ratios are computed, the advertisements are sorted in descending order based on this ratio. The algorithm then iteratively selects advertisements starting from the highest ratio, adding them to the selection as long as the total cost does not exceed the given budget. This process continues until no more ads can be added without violating the budget constraint.

The Greedy approach is highly efficient, with a time complexity of $O(n \log n)$ due to the sorting step. However, its main limitation is that it makes decisions based only on immediate benefits and does not consider future possibilities. As a result, it may fail to produce the optimal solution in certain cases, especially when a combination of lower-ratio items yields a higher total profit.

Greedy Approach

- Calculate:
ratio = profit / cost
- Sort ads in descending order of ratio
- Select ads until budget is exhausted

DYNAMIC PROGRAMMING APPROACH (0/1 Knapsack) :

The Dynamic Programming (DP) approach is used in this project to guarantee an optimal solution to the advertisement selection problem. It is based on the well-known 0/1 Knapsack Problem, where each item (advertisement) can either be selected completely or not selected at all.

$$dp[i][w] = \max(dp[i-1][w], profit_i + dp[i-1][w - cost_i])$$

This approach works by constructing a two-dimensional table $dp[i][w]$, where:

- i represents the number of advertisements considered
- w represents the current budget limit

Each cell in the table stores the maximum profit achievable using the first i ads within budget w . For every advertisement, the algorithm evaluates two choices:

1. Exclude the ad $\rightarrow$ profit remains the same as the previous state
2. Include the ad $\rightarrow$ profit increases, but the remaining budget decreases

By comparing these two options, the algorithm ensures that the best possible decision is made at every step. Unlike the Greedy method, Dynamic Programming explores all possible combinations systematically and stores intermediate results to avoid redundant calculations.

The time complexity of this approach is $O(n \times \text{budget})$, as it fills a table of size $n \times \text{budget}$. Similarly, the space complexity is also $O(n \times \text{budget})$ due to the storage of the DP table. Although

this method is computationally more intensive than the Greedy approach, its major advantage is that it always produces the optimal solution, making it highly reliable for decision-making problems.

WORKING EXAMPLE :

To understand how the system works, consider a simple example where the total available budget is 15 units, and there are two advertisements:

- Ad A → Cost = 10, Profit = 5
- Ad B → Cost = 5, Profit = 2

In the Greedy approach, the system calculates the profit-to-cost ratio and selects advertisements based on the highest ratio first. Depending on the ratio values, the selection order may vary, and while it often gives a good result, it does not guarantee the best possible outcome in all scenarios.

In contrast, the Dynamic Programming approach evaluates all possible combinations of advertisements. It checks:

- Selecting only A
- Selecting only B
- Selecting both A and B

After evaluating all combinations, it determines that selecting both advertisements results in a total cost of 15 and a total profit of 7, which is the optimal solution. This example clearly demonstrates how DP ensures accuracy by considering every possibility rather than making locally optimal choices.

TESTING & VALIDATION :

The system has been thoroughly tested to ensure correctness, reliability, and performance. Testing was conducted using datasets of varying sizes to evaluate how the algorithms behave under different conditions.

For small datasets (2–5 ads), the system was tested manually to verify that both Greedy and Dynamic Programming approaches produce correct outputs. These tests helped in validating the logic and ensuring that the implementation works as expected.

For medium datasets (10–20 ads), the system was tested to analyze performance and efficiency. In these cases, the difference between Greedy and DP results becomes more noticeable, especially when the Greedy approach fails to produce the optimal solution.

Several validation checks were performed during testing:

- Ensuring that the total cost of selected ads never exceeds the budget
- Verifying that the profit calculation is accurate
- Comparing outputs of both algorithms to confirm correctness
- Manually cross-checking results for smaller inputs

The testing process confirms that the Dynamic Programming approach consistently provides the optimal solution, while the Greedy approach offers faster but sometimes suboptimal results. This validates the effectiveness of the system and highlights the practical importance of algorithm selection.

COMPARISON OF ALGORITHMS :

Feature	Greedy	Dynamic Programming
Approach	Local optimal choice	Global optimal solution
Speed	Fast	Slower
Accuracy	Not guaranteed	Always optimal
Complexity	$O(n \log n)$	$O(n \times W)$

CODE STRUCTURE EXPLANATION :

```
typedef struct {
    char name[50];
    int cost;
    int profit;
    float ratio;
} Ad;
```

- Stores advertisement details.

Sorting Function

- Uses `qsort()`
- Sorts ads based on ratio

Greedy Function

- Iterates through sorted ads
- Adds ads until budget limit

DP Function

- Creates 2D array `dp[n][budget]`
- Backtracks to find selected ads

Key Concepts Used

- Greedy Method
- Dynamic Programming
- 0/1 Knapsack Problem
- Sorting Algorithms
- Structures in C
- Memory Optimization

Real-World Applications

- Digital marketing budget optimization
- Resource allocation problems

- Investment planning
- Project selection systems
- Logistics and supply chain optimization

LIMITATIONS :

Despite its effectiveness, the Smart Ad Optimization System has certain limitations that should be considered. The Dynamic Programming approach, while guaranteeing optimal results, requires significant memory and computational resources, especially when the budget value is large. Since it uses a two-dimensional table of size $n \times \text{budget}$, the space and time requirements can grow rapidly, making it less efficient for very large inputs.

On the other hand, the Greedy algorithm, although fast and efficient, does not always produce the optimal solution. It makes decisions based on local optimal choices (profit-to-cost ratio) without considering future possibilities, which can lead to suboptimal results in certain scenarios.

Additionally, the current system operates on static, user-provided data and does not integrate with real-time advertising platforms. This limits its applicability in dynamic environments where ad performance metrics change frequently. The system also lacks persistent storage and advanced user interfaces, which could improve usability and scalability.

FUTURE ENHANCEMENTS :

The project can be significantly improved and extended in several ways to make it more practical and industry-ready. One major enhancement would be the addition of a Graphical User Interface (GUI), which would improve user interaction and make the system more visually appealing and easier to use compared to the current text-based interface.

Another important improvement is the integration of real-time ad analytics, where the system can fetch live data such as click-through rates, impressions, and conversion rates. This would allow for more accurate and dynamic decision-making.

The system can also be enhanced by incorporating Machine Learning techniques to predict advertisement profits based on historical data. This would transform the project from a rule-based system into an intelligent decision-making tool.

Furthermore, converting the application into a web-based platform would make it accessible from anywhere and allow multiple users to interact with the system simultaneously. Adding database support would enable storage of advertisement data, user inputs, and results for future analysis.

Additional improvements could include:

- Optimization for large-scale datasets
- Integration with cloud services
- Multi-budget and multi-campaign planning features

ADVANTAGES OF THE SYSTEM :

The Smart Ad Optimization System offers several advantages that make it both educational and practical. One of its key strengths is its simplicity and ease of use, as users can easily input data and obtain results without requiring advanced technical knowledge.

The system effectively demonstrates core algorithmic concepts, particularly Greedy and Dynamic Programming techniques, making it a valuable learning tool for students studying Design and Analysis of Algorithms. By implementing both approaches, the system provides a clear understanding of how different algorithms behave under the same conditions.

Another major advantage is its ability to provide a comparative analysis between two algorithmic strategies. This helps users understand the trade-offs between speed and accuracy, which is a crucial aspect of algorithm design.

The system is also flexible, allowing users to input different numbers of advertisements and budget values, making it adaptable to various scenarios. Additionally, the structured design and modular code make it easy to extend and modify for future improvements.

CONCLUSION :

The Smart Ad Optimization System successfully demonstrates the application of algorithmic techniques to solve a practical and relevant real-world problem. By modeling advertisement selection as an optimization problem, the project highlights how computational methods can significantly improve decision-making in constrained environments.

The implementation of the Greedy algorithm provides a fast and efficient solution, making it suitable for scenarios where quick decisions are required. However, its limitation in guaranteeing optimal results emphasizes the need for more robust approaches.

The Dynamic Programming approach, based on the 0/1 Knapsack problem, ensures that the best possible solution is always achieved by evaluating all combinations. Although it requires more computational resources, its accuracy makes it highly reliable for critical applications.

Overall, this project emphasizes the importance of choosing the appropriate algorithm based on the problem requirements, constraints, and available resources. It also bridges the gap between theoretical concepts and real-world applications, making it a valuable learning experience and a strong foundation for future developments in optimization systems.
